Task A

A.1:

The guest lecturers chosen for this task are Dr. Arvind Sharma and Debashish Ghose. Debashish Ghose's lecture, talked about IoT protocols, with the focus on MQTT and other protocols such as CoAP and AMQP. He has in my opinion helped give an understanding of how MQTT works, and made the principle behind it more understandable. He explained how MQTT works for sending and receiving messages with helpful diagrams or figures that really helped with the visualization of how they work. He also discussed MQTT QoS levels that ensure reliable message delivery and the differences between them, which I think helped with knowing which QoS level to choose in the future in terms of how much a loss of message one can tolerate. He also discussed MQTT-SN as an alternative, the types of MQTT-SN gateways and the advantages of using it. He also compared MQTT to other protocols like CoAP and AMQP, showing how each has its strengths depending on the situation (Ghose, 2024). This lecture helped me understand how secure communication works in IoT systems, its importance and how different protocols can be used to meet specific needs.

Dr. Arvind Sharma's lecture focused on hardware security in IoT devices. He explained that hardware is the foundation for secure communication and computation. Dr. Sharma discussed threats like physical tampering, side-channel attacks, and risks in the supply chain, and he highlighted ways to address these issues, such as using tamper-proof designs and securing the supply chain. He also talked about how reverse engineering can be used to detect vulnerabilities in hardware (Sharma, 2024). This lecture helped me understand how important hardware is for IoT security and connected well with the course topics, like designing secure IoT systems and protecting hardware from attacks.

These two lectures have given me a better understanding of two important parts of IoT security: protecting the hardware and ensuring that there is secure communication. These were things that I did not think of as having such a huge importance in IoT prior to the lectures. It does make sense afterwards that these are both important parts of IoT security, because if the hardware and the communication are not protected, they will be vulnerable to attacks which is not something that we would want at all. These two things are what I feel are usually taken for granted as we don't

really think about these things, but due to these lectures the awareness of protecting and ensuring that they are secure will definitely be with me when working with IoT in the future.

A.2:

The Radio Frequency Spectrum Analyzer is a device that can listen to and monitor frequencies ranging from 0 to 60 GHz. It is capable of analyzing used channels, such as those in 2.4 GHz Wi-Fi networks. This simple device measures the strength of signals, their levels, and their frequencies, and can also identify the types of protocols in use. It can detect unconventional frequencies, such as walkie-talkie transmissions at 146 MHz. The closer the analyzer is to the transmitting antenna, the stronger the signal it detects, while distance weakens the signal strength. This device can identify ongoing transmissions, making it a valuable tool for IoT security (Shalaginov, 2024, Session 6).

My understanding is that the Radio Frequency Spectrum Analyzer can be used to assess the security of generic IoT smart home applications by identifying unauthorized devices or frequencies. It allows users to point out frequencies that are not authorized and are potentially targeting smart home systems. The analyzer can also detect weak protocols and monitor Wi-Fi signals for vulnerabilities. By identifying discrepancies in signal strength, levels, and frequencies, it becomes possible to monitor and mitigate potential threats to IoT systems. Additionally, this device can strengthen security by identifying gateways and channels where unauthorized communication attempts are occurring.

The Software Defined Radio or SDR is a flexible and versatile tool that can listen to frequencies ranging from 0 to 235 GHz (Shalaginov, 2024, Session 7). SDRs can record signals and analyze their protocols, and can even listen to frequencies from the International Space Station (Shalaginov, 2024, Session 7). Unlike traditional radios, SDRs use software-based components to process digital signals, allowing them to adapt to various frequencies and protocols with a software update. SDRs can identify By analyzing signal strength, modulation, and frequency, SDRs can identify unauthorized transmissions and inspect protocols for vulnerabilities.

My understanding is that SDRs are an essential tool for security assessment in generic IoT smart home applications. They can help identify unauthorized devices or signals within the wireless spectrum, making them valuable for detecting weak or unencrypted protocols, pinpointing signal anomalies, and monitoring unusual communication attempts. By analyzing discrepancies in

frequencies or signal strength, SDRs help uncover vulnerabilities, providing insights in order to secure gateways and connected devices. The adaptability and versatility that SDRs have are important in safeguarding IoT ecosystems.

Task B

B.1:

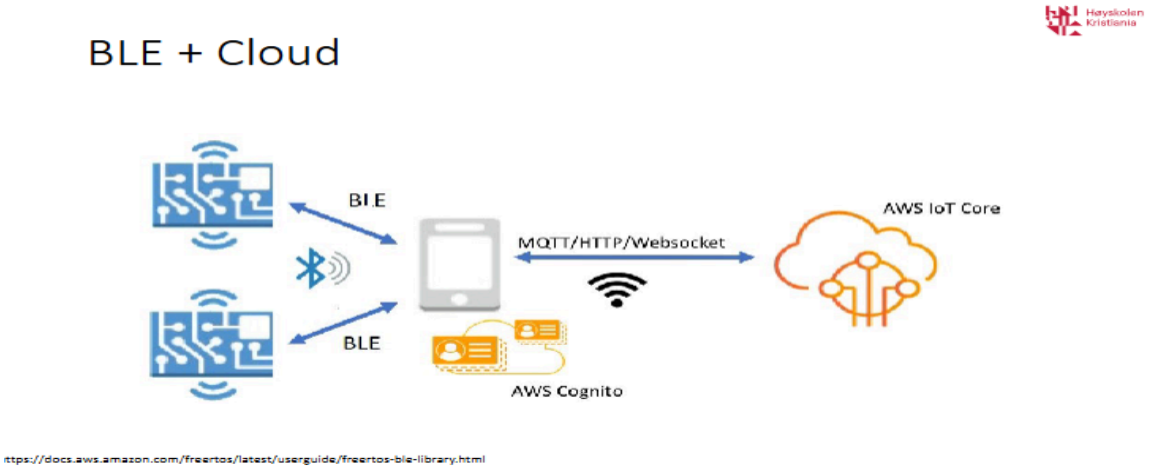

Figure B.1: Lecture 9, Slide 96 Generic BLE + Cloud application integration implementation

To use threat modeling in the security assessment of this diagram, the STRIDE framework will be followed as it focuses on identifying weaknesses in the technology (Chantzis et al. , 2021).
The STRIDE framework has three steps, which will be listed below.

1. Identify the architecture:

   This first step examines the device's architecture. From the diagram provided, it seems like the device's architecture contains the following:

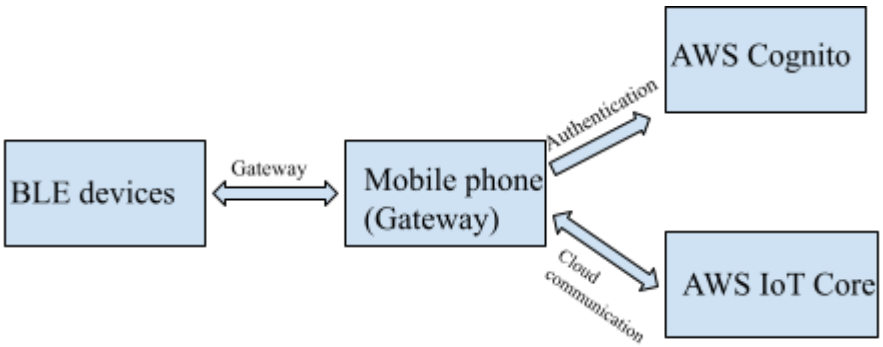

   Figure B.2 Simplified breakdown of the architecture.

- BLE devices that connects via bluetooth
- A gateway, such as smart phone that connects to the cloud via MQTT/HTTP/Websocket
- AWS Cognito, which is a service to secure customer identity and access management (Amazon Web Services, n.d)
- AWS IoT core, which provides cloud services that connects IoT pieces to other devices and AWS cloud services (Amazon Web Services, n.d)

2. Break into components

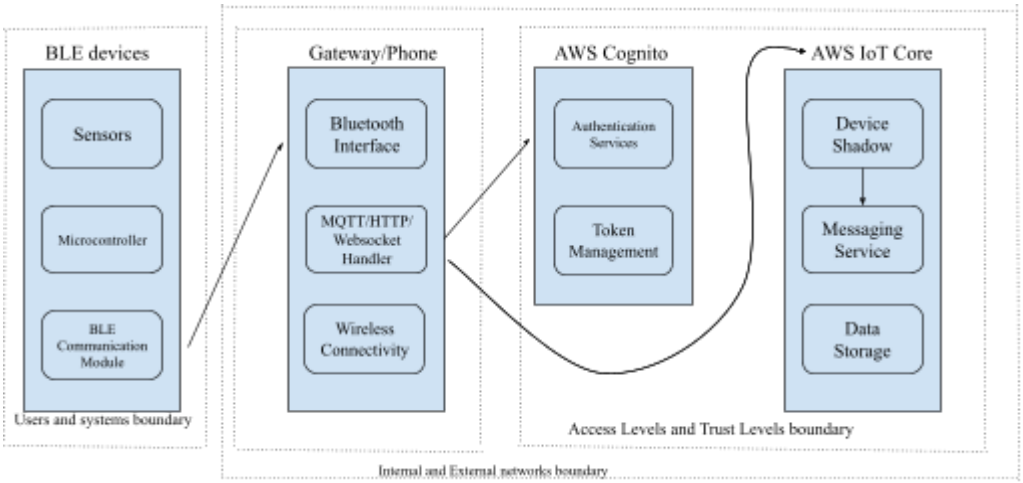

Figure B.3 Breakdown of the components with trust boundaries

From the architecture, each component has been further broken down, and put into trust boundaries as well.

The trust levels boundaries are grouped as users and systems boundary, internal and external networks boundary and access levels and trust levels boundary.

In the users and systems boundary, the BLE devices interact with the gateway or the phone, where we go from a low-trust user environment to a higher-trust system. The BLE devices include sensors, microcontrollers and BLE communication modules and collect and transmit data via BLE protocols (BasuMallick, 2022). The sensors collect environmental or system data, while the microcontroller controls signal broadcasting (GAO Tek, n.d.). The BLE communication module connects with the Bluetooth interface of the gateway, which acts as the entry point between the two components and ensures that there is a secure communication between the devices (Andersen 2024).These components are considered untrusted because they operate in user-facing environments where tampering or spoofing can occur (GAO Tek, n.d, Andersen, 2024). It also is

the connection point between users and systems boundary and the internal and external networks boundary.

The Gateway is responsible for bridging the connection between the BLE devices and AWS services. The gateway, which is an internal network, is grouped within the internal and external networks boundary together with AWS Cognito and AWS IoT core, which are external networks. The gateway is broken down into a bluetooth interface, MQTT/HTTP/websocket handler and wireless connectivity. The bluetooth interface enables communication with nearby devices and ensures secure data transfer to and from the BLE devices (GAO Tek, n.d.). The MQTT/HTTP/Websocket handler manages cloud communication by transmitting data to AWS Cognito and AWS IoT core using internet protocols (Amazon Web Services, n.d.). Wireless connectivity establishes internet access via Wi-Fi or cellular connection to connect to the AWS services (Amazon Web Services, n.d.). The data moves between the gateway and the AWS services via the MQTT/HTTP/Websocket handler, encryption protects the information during transfer, while authentication such as MFA validates the identity of the gateway and the users (Andersen, 2024). The boundary tells the need for robust encryption and authentication to secure the connection between the internal and external systems.
AWS IoT core and the AWS Cognito are grouped under access levels and trust levels boundary, which ensures that only authenticated and authorized devices or users can access the cloud services. AWS Cognito has two components: authentication services and token management. Authentication services validates the identity of the users or the devices before it allows access to the cloud services (Amazon Web Services, n.d). Token management on the other hand issues secure tokens to authenticated users or devices for access to cloud resources (Amazon Web Services, n.d). The AWS IoT core has three components, device shadows, messaging services and data storage. Device shadow maintains a virtual representation of the current and last known states of IoT devices (Amazon Web Services, n.d). Messaging services route messages between devices and users or applications (Amazon Web Services, n.d). Data storage stores device-generated data securely for analysis, retrieval and monitoring (Amazon Web Services, n.d).

3. Identify threats to each component

To identify threats to each component, we use:
Spoofing, **T**ampering, **R**epudiation, **I**nformation Disclosure, **D**enial of Service and **E**levation of Privilege, which each component will have listed below (Chantzis et al. , 2021):

BLE Devices

- Sensors:
    - **Spoofing**: impersonating sensors.
    - **Tampering**: modifying sensor firmware.
    - **Repudiation**: no secure logs for activity.
    - **Information Disclosure**: unencrypted data exposed.
    - **DoS**: excessive commands stop sensors.
    - **Elevation of Privilege**: exploiting sensor vulnerabilities.
- Microcontroller:
    - **Spoofing**: impersonating microcontroller.
    - **Tampering**: modifying firmware.
    - **Repudiation**: insecure or missing logs.
    - **Information Disclosure**: exposed operational data.
    - **DoS**: overload disrupts functions.
    - **Elevation of Privilege**: firmware exploit.
- BLE Communication Module:
    - **Spoofing**: impersonating BLE module.
    - **Tampering**: intercepting or altering communications.
    - **Repudiation**: no logs for interactions.
    - **Information Disclosure**: exposed BLE transmissions.
    - **DoS**: connection floods disrupt communication.
    - **Elevation of Privilege**: BLE protocol exploit.

Gateway

- Bluetooth Interface:
    - **Spoofing**: fake BLE devices connect.
    - **Tampering**: modifying Bluetooth stack.
    - **Repudiation**: logs missing or insecure.
    - **Information Disclosure**: unencrypted Bluetooth data exposed.
    - **DoS**:overwhelm connection floods .
    - **Elevation of Privilege**: Bluetooth exploit
- MQTT/HTTP/WebSocket Handler:
    - **Spoofing**: impersonating clients.

    - **Tampering**: altering messages.
    - **Repudiation**: no logs for protocol activity.
    - **Information Disclosure**: unencrypted messages.
    - **DoS**: excessive requests block communication.
    - **Elevation of Privilege**: protocol handler exploit.
- Wireless Connectivity:
    - **Spoofing**: fake access points.
    - **Tampering**: altering wireless configurations.
    - **Repudiation**: logs missing or insecure.
    - **Information Disclosure**: unencrypted wireless traffic exposed.
    - **DoS**: jamming signals.
    - **Elevation of Privilege**: wireless protocol exploit.

AWS Cognito

    - **Spoofing**: impersonating users/devices.
    - **Tampering**: altering tokens.
    - **Repudiation**: insecure authentication logs.
    - **Information Disclosure**: exposed credentials.
    - **DoS**: excessive authentication requests.
    - **Elevation of Privilege**: compromised accounts.

AWS IoT Core

- Device Shadow:
    - **Spoofing**: fake devices update states.
    - **Tampering**: altering shadow states.
    - **Repudiation**: insecure logs for state updates.
    - **Information Disclosure**: exposed device state data.
    - **DoS**: shadow request floods.
    - **Elevation of Privilege**: unauthorized state updates.
- Messaging Service:
    - **Spoofing**: fake devices send messages.
    - **Tampering**: altering routed messages.
    - **Repudiation**: no logs for messaging.

    - **Information Disclosure**: exposed unencrypted messages.
    - **DoS**: message floods.
    - **Elevation of Privilege**: API exploit.
- Data Storage:
    - **Spoofing**: impersonating applications.
    - **Tampering**: modifying stored data.
    - **Repudiation**: insecure logs for data access.
    - **Information Disclosure**: exposed unencrypted data.
    - **DoS**: excessive storage requests.
    - **Elevation of Privilege**: storage API exploit.

B.2:

Two possible attack scenarios will be listed below that will base itself on B1, using the DREAD risk-based framework that answers the following questions below and will be scored between 0-10 according to their threats (Chantzis et al. , 2021):

- **D**amage: How damaging the exploitation of this threat would be
- **R**eproducibility: How easy the exploit is to reproduce
- **E**xploitability: How easy the threat is to exploit
- **A**ffected User: How many users would be affected
- **D**iscoverability: How easy it is to identify the threat

Attack Scenario 1: Signal Jamming Attack

- In this scenario, communication between BLE devices, such as the sensors and the Gateway are disrupted by jamming their communication signal. This isolates BLE devices from the system and can result in communication failures or operational delays.

Threat	**Score**
Damage	9
Reproducibility	8
Exploitability	7

Affected Users	9
Discoverability	8
Total	8.2

Overall: 8.2 out of 10 - high risk

Attack Scenario 2: Spoofing a BLE device

In this attack scenario, an attacker impersonates a BLE device by spoofing its identity and communicates with the Gateway as if it were a legitimate device. This could allow the attacker to send falsified data, disrupt operations, or gain unauthorized access to the system.

Threat	**Score**
Damage	8
Reproducibility	8
Exploitability	6
Affected Users	8
Discoverability	7
Total	7.4

Overall: 7.4 out of 10 - high risk

Task C

C.1:

A simple MQTT-based IoT application that includes three components made in a Docker instance has been made:

- A plant/flower water and monitoring station, which is a simulated IoT device that acts as a virtual soil humidity sensor, publishing data to a central MQTT broker such as Mosquitto.
- A server-based cloud solution that aggregates the data, stores it in a MySQL database, and provides API endpoints via a Flask application

- A user-facing mobile-friendly client application built with HTML/JavaScript that visualizes historical soil humidity trends.

The diagram of the designated system where the system components are as follows:

- IoT device: A python app that sends the humidity data via MQTT
- Central broker: Mosquitto in charge of communication between IoT devices and the database
- Database: stores historical soil humidity data
- API server: Retrieves data from the database and provides to the client app
- Client Application: Displays historical soil humidity data to the user.

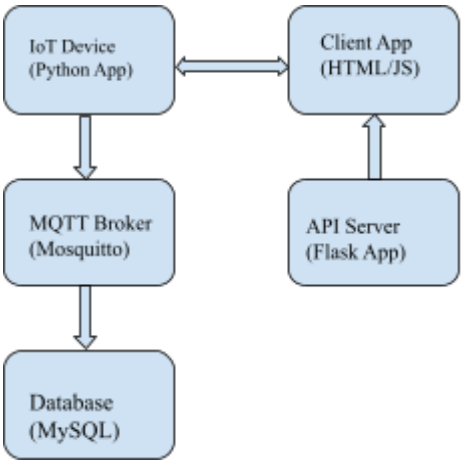

Figure C.1 Diagram of the MQTT-based IoT application

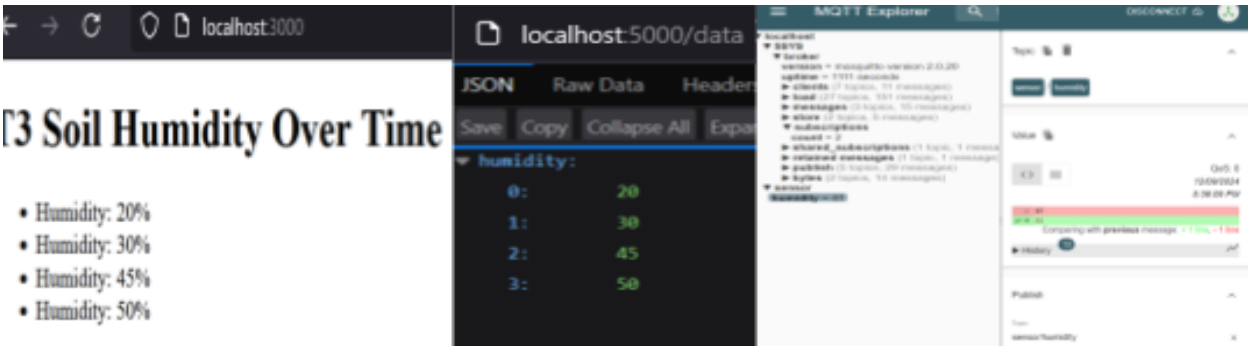

Figure C.2 Visualization of historical soil humidity trends

Figure C.3 shows snippets of the codes from the IoT device code and the MQTT broker configuration. The main.py in the IoT Device Code stimulates a sensor that generates random humidity values between 10 and 90. Using the Python paho-mqtt library, it connects to the MQTT broker hosted on localhost and publishes these values to the topic sensor/humidity at regular intervals, every 5 seconds. The code demonstrates the core functionality of the IoT device, which is to mimic real-world sensor behavior and enable communication with the MQTT broker.

The MQTT broker configuration file- mosquitto.conf, sets up a listener on port 1883 and allows for anonymous connections. This configuration ensures the broker can accept messages from the IoT device and forward them to any subscribers.

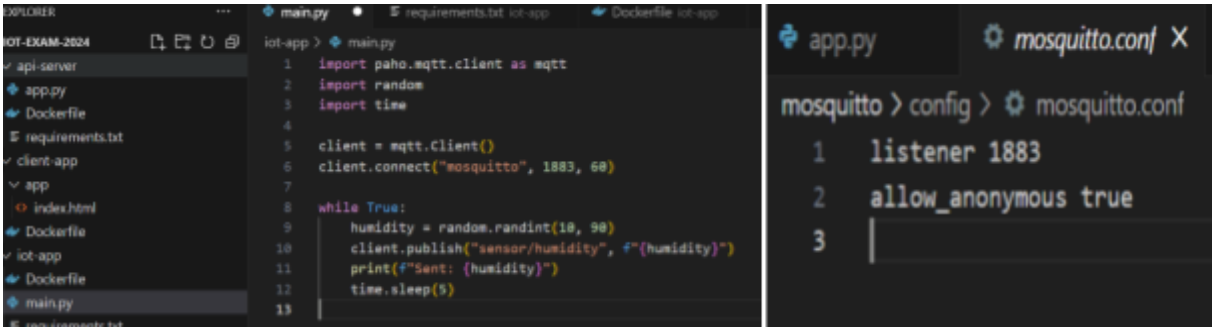

Figure C.3 Snippets of the codes from the IoT device code and the MQTT broker configuration

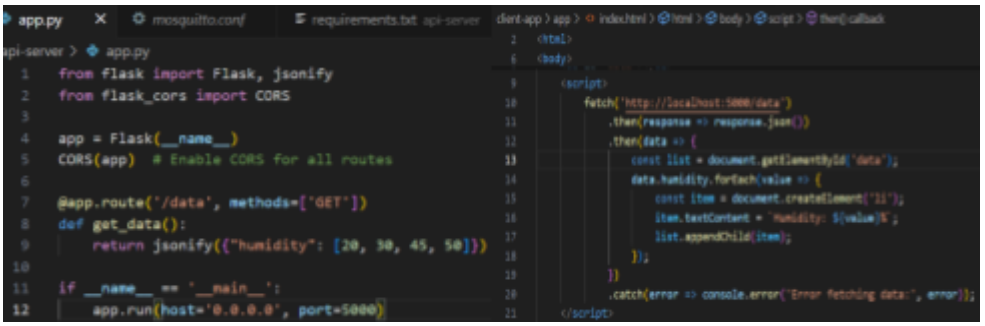

Figure C.4 Snippets from the API server code and Client Application code.

The API server code shown in Figure C.4, is a Flask application that retrieves data from the MySQL database and serves it to the client via a RESTful /data endpoint. It uses the mysql.connector library to establish a connection to the database and fetches up to 10 records of humidity data. Basic Cross-Origin Resource Sharing or CORS is enabled to allow the client application to access the API. This code snippet highlights the role of the API server as a bridge between the database and the client, ensuring seamless data access.

The client app code also shown in Figure C.4, is a JavaScript snippet embedded in the front-end application, which fetches humidity data from the API- http://localhost:5000/data and dynamically displays it on a webpage. This snippet illustrates how the client app interfaces with the API to provide users with a visual representation of the sensor data.

Some possible vulnerabilities and weaknesses of the system includes:

- unencrypted MQTT communication as it transmits data in plaintext, and is vulnerable to interception that can lead to a man-in-the-middle attack. The data can be intercepted and

modified causing harm. In this case, watering too much will cause the flowers/plants to die due to high soil humidity.

- insecure broker access (no authentication), allows anonymous connections by default so that anyone can publish or subscribe to topics without providing credentials. Without authentication, an attacker can intercept sensitive data.
- potential SQL injection in the API server if the server takes user inputs. This can mean that the attackers can retrieve data from the database by using query conditions. They can delete or modify the data.

C.2:

The project chosen for this task is : MP3 player control with webserver using ESP32 WIFI from https://projecthub.arduino.cc/carlosvolt/mp3-player-control-with-webserver-using-esp32-wifi-268718
The project was chosen after searching for ESP32 webserver on https://projecthub.arduino.cc and was the only one that was for a webserver and via wi-fi.

The setup () handles all initialization tasks such as establishing communication with the serial interfaces, connection to wi-fi and setting up the web server and defining its routes. The loop () is empty because the asynchronous web server manages the client interactions.

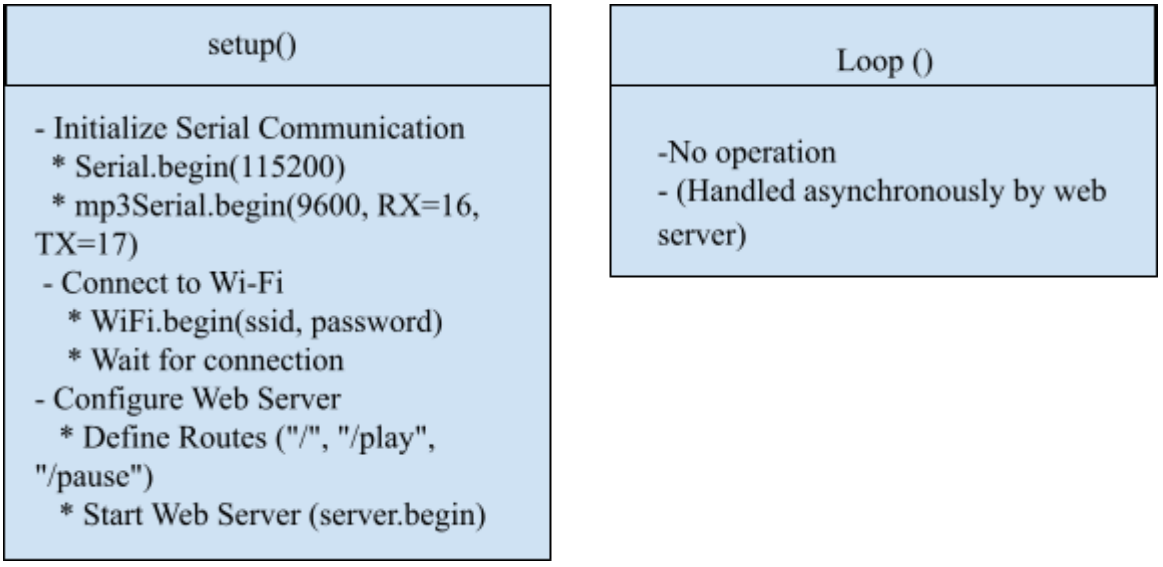

Figure C.5 a simple diagram of functionality implemented in setup() and loop() procedures

The project will be analyzed with a list of their corresponding Devices Properties, Threats and Mitigations according to MITRE EMB3D Threat Model Framework (MITRE 2024). The categories have their corresponding identifiers and descriptions from EMB3D included.

Device Properties:

- DEV-001: Device Type – ESP32 microcontroller with Wi-Fi capabilities.
- DEV-002: Interfaces – UART for communication with the YX5300 MP3 module; Wi-Fi for network connectivity.
- DEV-003: Components – YX5300 MP3 module; ESPAsyncWebServer library for web server functionality.
- DEV-004: Functions – Remote control of MP3 playback (play, pause, stop, next, previous, volume control) via a web interface.
- DEV-005: Communication Protocols – HTTP for web server interactions; custom serial protocol for MP3 module commands.

Threats

- THR-001: Unauthorized Access – The web server lacks authentication, allowing any networked device to control the MP3 player.
- THR-002: Data Interception – HTTP communication is unencrypted, making sensitive data vulnerable to interception.
- THR-003: Command Injection – Improper input handling could allow unauthorized execution of commands on the MP3 module.
- THR-004: Denial of Service (DoS) – Excessive requests could overwhelm the server, causing it to become unresponsive.
- THR-005: Weak Wi-Fi Security – Weak passwords or spoofing attacks could compromise the device's Wi-Fi connection.

Mitigations

- MIT-001: Authentication – Implement authentication mechanisms (e.g., requiring login credentials or tokens) to restrict access.
- MIT-002: Encryption – Use HTTPS to encrypt HTTP communication and prevent data interception.

- MIT-003: Input Validation – Sanitize inputs and validate all commands to mitigate command injection risks.
- MIT-004: Rate Limiting – Implement rate limiting and monitor traffic patterns to prevent DoS attacks.
- MIT-005: Wi-Fi Security – Enforce WPA2/WPA3 encryption and require strong, unique passwords for the network.

The MITRE EMB3D Threat Model provides a structured approach to improving IoT security by identifying device properties, mapping them to threats, and applying appropriate mitigations. It emphasizes analyzing the device's hardware, software, and communication protocols to uncover vulnerabilities. Threats are evaluated for their relevance and impact, and mitigations are categorized as foundational , intermediate, or advanced. By addressing these threats with proper mitigations, IoT security aligns with EMB3D's best practices, ensuring protection against evolving risks (MITRE, 2024).

Reference List:

Amazon Web Services (AWS). (n.d.). *Amazon Cognito*. Retrieved December 7, 2024, from https://aws.amazon.com/pm/cognito/

Amazon Web Services (AWS). (n.d.). *AWS IoT Core documentation*. Retrieved December 7, 2024, from https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html

Andersen, L. R. (2024, September 26). *What is a trust boundary?* Retrieved December 7, 2024, from https://moxso.com/blog/what-is-a-trust-boundary#:~:text=Simply%20put%2C%20a%20trust%20boundary

BasuMallick, C. (2022, June 10). *What is Bluetooth LE?* Spiceworks. Retrieved December 7, 2024, from https://www.spiceworks.com/tech/iot/articles/what-is-bluetooth-le/

Chantzis, F., Stais, I., Calderon, P., Deirmentzoglou, E., & Woods, B. (2021). *Practical IoT hacking: The definitive guide to attacking the Internet of Things*. No Starch Press.

GAO Tek. (n.d.). *Structure and components of a BLE system: Gateways, beacons, and accessories*. Retrieved December 7, 2024, from https://gaotek.com/structure-and-components-of-a-ble-system-gateways-beacons-and-accessories/

Ghose, D. (2024). *IoT protocols* [Guest lecture]. Kristiania University College.

MITRE. (2024). *MITRE EMB3D™ Threat Model*. Retrieved from https://emb3d.mitre.org/

OWASP Foundation. (n.d.). *OWASP Internet of Things Project*. Retrieved [December 9, 2024], from https://wiki.owasp.org/index.php/OWASP_Internet_of_Things_Project

Shalaginov, A. (2024). *IOS3100 – Session 2: Introduction to components, vulnerabilities, and countermeasures* [Lecture slides]. Kristiania University College

Shalaginov, A. (2024). *IOS3100 – Session 6: Interfaces, hardware hacking, and RTOS* [Lecture slides]. Kristiania University College.

Shalaginov, A. (2024). *OS3100 – Session 7: Firmware hacking, drones, Python for Arduino* [Lecture slides]. Kristiania University College

Sharma, A. (2024). *Importance of hardware security in embedded systems* [Guest lecture]. Kristiania University College.